

# Small-Perturbation Adversarial Example Detection via Proactive Decision Boundary Bending

Nankun Mu , Qi Xia, Fengyi Jiang, Hongyu Huang , *Member, IEEE*, Di Zhang , Xiaowei Yang, Bei Gong , *Member, IEEE*, Weizhi Meng , *Senior Member, IEEE*, and Xinyu Lei , *Member, IEEE*

**Abstract**—Deep neural networks (DNNs) are highly susceptible to adversarial example (AE) attacks, posing serious security threats. As a specific class of adversarial attacks, small-perturbation adversarial (SPA) attacks have emerged as a more severe threat to DNNs due to their subtle perturbation magnitudes, making them harder to detect. However, there is still a lack of specific solutions that are designed to defend against SPA attacks. To fill in this gap, we propose Proactive Decision Boundary Bending (PDBB) for SPA example detection in this article. Via proactively bending the decision boundary of the target DNN, PDBB can accurately distinguish between SPA examples and benign examples. Specifically, PDBB leverages the example-perturbation training strategy to train the target DNN, enabling it to possess desirable bended decision boundaries. In addition, PDBB develops the projection-based auxiliary measurement method that accurately measures the concavity-convexity of the decision boundary in the vicinity of a given test example, thereby enabling PDBB to achieve high detection accuracy. Extensive experiments demonstrate that PDBB not only achieves the state-of-the-art (SOTA) performance in detecting SPA examples but also achieves high performance in detecting general AEs (that may have relatively larger perturbations).

**Index Terms**—Deep neural networks, adversarial example detection, concavity-convexity measurement.

## I. INTRODUCTION

DEEP neural networks (DNNs) have played a crucial role in various daily tasks, such as autonomous driving [3], [23], medical diagnosis [11], and face recognition [8], [47] due to their powerful feature extraction and pattern recognition capabilities. However, DNNs are vulnerable to adversarial examples (AEs) [5], [14], [34], [37], which pose severe

threats to these safety-critical tasks. An adversarial example is a specially crafted input to a DNN model that has been intentionally perturbed to cause the model to make an incorrect prediction [46]. For example, in autonomous driving, adversarial attacks on camera-captured images can cause the driving system to misclassify a stop sign as a speed-up sign, while human observers remain unaware of any abnormalities.

As a specific class of adversarial attacks, small-perturbation adversarial (SPA) attacks [1], [40] allow an attacker to gradually seek a small perturbation through optimization methods such that the perturbed AEs can be misclassified. Since the perturbation magnitude is small, the generated AEs are harder to detect by humans. For example, the most typical SPA attack is the Fast Minimum-Norm attack [6], which aims to mislead classification by finding the smallest input perturbation according to a given norm. Compared with other adversarial attacks, SPA attacks are more stealthy and pose more serious threats to DNN models. As a result, it is imperative to design solutions to defend against SPA attacks.

**Limitations of Prior Art:** Various solutions have been proposed to defend against adversarial attacks. The previous defenses can be classified into the non-detection-based solutions and the detection-based solutions. The non-detection-based solutions employ the adversarial training strategy [42], [43], [49] and the input purification strategy [21], [41]. One key limitation of non-detection-based solutions is that they need to sacrifice model accuracy to achieve a strong defense capability. The detection-based solutions [15], [29], [32] mainly focus on detecting general AEs (that may have relatively larger perturbations), whereas their detection accuracy on SPA examples is unsatisfactory. Unlike previous solutions, this paper aims to develop the first specific solution to detect SPA examples. Moreover, the prior detection-based solutions are usually passive, indicating that no modifications are performed on the target DNN model to facilitate the AE detection. Unlike prior detection-based solutions, we develop an active solution by proactively bending the decision boundary of the target DNN model to detect SPA examples more precisely.

**Proposed Approach:** In this paper, we propose PDBB for SPA example detection via proactive decision boundary bending. Fig. 1 illustrates our key idea for SPA example detection. For the target DNN, the defender can actively bend its decision boundaries in the model training phase. Once the SPA examples are created along the shortcut, they face a concave decision boundary in their vicinity. In contrast, the benign examples face

Received 26 January 2026; revised 18 June 2026; accepted 19 June 2026. Date of publication 25 June 2026; date of current version 1 September 2026. This work was supported by the National Natural Science Foundation of China under Grant 62272073, Grant 62322601, and Grant 62302072. (Corresponding author: Bei Gong.)

Nankun Mu, Qi Xia, Fengyi Jiang, Hongyu Huang, and Di Zhang are with the College of Computer Science, Chongqing University, Chongqing 400044, China (e-mail: nankun.mu@cqu.edu.cn; 202314131035t@stu.cqu.edu.cn; felixjfy@hotmail.com; hyhuang@cqu.edu.cn; dizhang@cqu.edu.cn).

Xiaowei Yang is with the China Industrial Control Systems Cyber Emergency Response Team, Beijing 100124, China (e-mail: xiaoweiyang@gmail.com).

Bei Gong is with the Beijing Key Laboratory of Trusted Computing, College of Computer Science, Beijing University of Technology, Beijing 100124, China (e-mail: gongbei@bjut.edu.cn).

Weizhi Meng is with the School of Computing and Communications, Lancaster University, LA1 4YW Lancaster, U.K. (e-mail: weizhi.meng@ieee.org).

Xinyu Lei is with the Department of Computer Science, Michigan Technological University, Houghton, MI 49931 USA (e-mail: xinyulei@mtu.edu).

Digital Object Identifier 10.1109/TDSC.2026.3707542

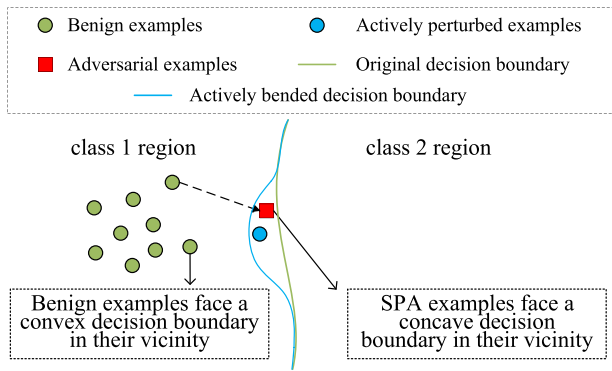


Fig. 1. Illustration of our key idea for SPA detection. In a model with actively bended boundaries, benign examples always face a convex decision boundary in their vicinity. On the contrary, SPA examples always face a concave decision boundary in their vicinity. This distinction enables the design of a detection method that leverages the differences in concavity-convexity. Note that the shortcut from class 1 to class 2 (for benign examples in class 1) and the shortcut from class 2 to class 1 (for benign examples in class 2) are in different locations. Therefore, the benign examples in class 2 will also face a convex decision boundary in their vicinity.

a convex decision boundary in their vicinity. Therefore, given a test example, the defender can measure the concavity-convexity of the decision boundary in its vicinity to identify whether it is an SPA example or a benign example.

**Challenges and Solutions:** There are three major technical challenges to be addressed.

1) It is challenging to accurately distinguish between SPA examples and benign examples since the added perturbation on the SPA examples is small. To address this challenge, we actively bend the decision boundary of the DNN, intending to amplify the curvature of the boundary. After the actively bending process, the differences in concavity-convexity of the decision boundary separating benign and SPA examples are amplified. As shown in Fig. 1, SPA examples tend to face a concave decision boundary in their vicinity, whereas benign examples tend to face a convex decision boundary. Therefore, the concavity-convexity can be used to accurately distinguish between SPA examples and benign examples.

2) It is a non-trivial task to achieve the desirable decision boundary bending while still maintaining the original task accuracy. To handle this issue, we develop the example-perturbation training strategy as illustrated below. First, for each class, the defender slightly perturbs a small portion of examples and assigns them different labels. This perturbation strategy is applied to all label pairs to ensure coverage of all potential attack cases. Then, the perturbed examples are inserted into the original dataset, which is used to train the DNN, thereby achieving the desirable decision boundary bending. Since the boundary bending is subtle, there is nearly no observable degradation in original task accuracy caused by our example-perturbation training strategy.

3) It is challenging to develop a method to accurately measure the concavity-convexity of the decision boundary. To tackle this challenge, we propose the projection-based auxiliary measurement method illustrated as follows. Given a test example, it is first projected onto the decision boundary. Then, the projected example is perturbed in two directions (that are opposite to each other) and two perturbed examples are obtained. Next, the

perturbed examples are re-projected onto the decision boundary to obtain a measurement point by calculating the midpoint of these corresponding projected examples. If the test example and the measurement point lie in two different classification regions, then the test example faces a convex decision boundary (i.e., it is a benign example). Otherwise, if the test example and the measurement point lie in the same classification region, then the test example faces a concave decision boundary (i.e., it is an SPA example). Please refer to Fig. 3 for details.

**Broader Applicability:** Although PDBB is developed to detect SPA examples, we note that it can work well for detecting general AEs (that may have relatively larger perturbations). Our experimental results show that 1) for SPA example detection, PDBB outperforms prior defenses (that are designed to detect general AEs); 2) for general AE detection, PDBB achieves performance comparable to prior defenses. Therefore, PDBB serves as a panacean solution to detect AEs, regardless of whether they are SPA examples or general AEs.

**Contributions:** The main contributions of this work are summarized as follows.

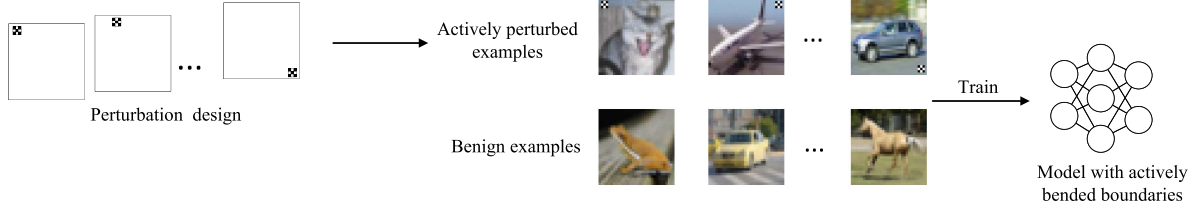
- We are the first to develop an SPA detection solution by proactively bending the decision boundary and then leveraging the differences in the concavity-convexity of the boundary between benign and SPA examples. In contrast, prior detection-based defenses are usually passive, indicating no modifications are performed on the target DNN to facilitate the AE detection.
- We develop the example-perturbation training strategy to train the target DNN. The trained DNN achieves the desirable decision boundary bending while still maintaining the original task accuracy.
- We develop the projection-based auxiliary measurement method that accurately measures the concavity-convexity of the decision boundary in the vicinity of a test example using projection and perturbation techniques.
- The developed PDBB not only achieves the SOTA performance in detecting SPA examples but also achieves high performance for detecting general AEs (that may have relatively larger perturbations), thereby achieving broader applicability.

## II. RELATED WORK

### A. Adversarial Attacks

Based on different generation methods, adversarial attacks can be roughly categorized into gradient-based attacks and optimization-based attacks. Gradient-based attacks [14], [34] focus on efficiency, generating adversarial samples using simple gradient direction update rules. Optimization-based attacks [5], [6], [37], [40] leverage an optimization process to find the optimal perturbation, achieving stronger attack performance. With the development of adversarial attacks, attackers pursue not only a high attack success rate but also a small adversarial perturbation. SPA attacks constitute a subset of adversarial attacks characterized by explicitly seeking minimum-norm perturbations that cross the decision boundary along the shortest path. In contrast, general attacks typically aim to generate successful adversarial examples within a predefined perturbation budget

## ① Actively bending phase



## ② AE detection phase

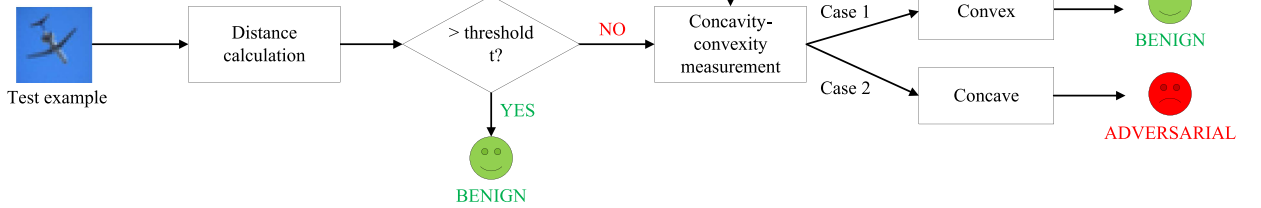


Fig. 2. Overview of PDBB. PDBB is composed of the actively bending phase and the AE detection phase. In the actively bending phase, a model with actively bended boundaries is trained using a combination of actively perturbed examples and benign examples. In the AE detection phase, we first calculate the minimum distance of the tested examples to the decision boundary and detect AEs using the distance. Then we measure the concavity-convexity of the decision boundary in the vicinity of the remaining examples and perform a second round of AE detection using this concavity-convexity feature.

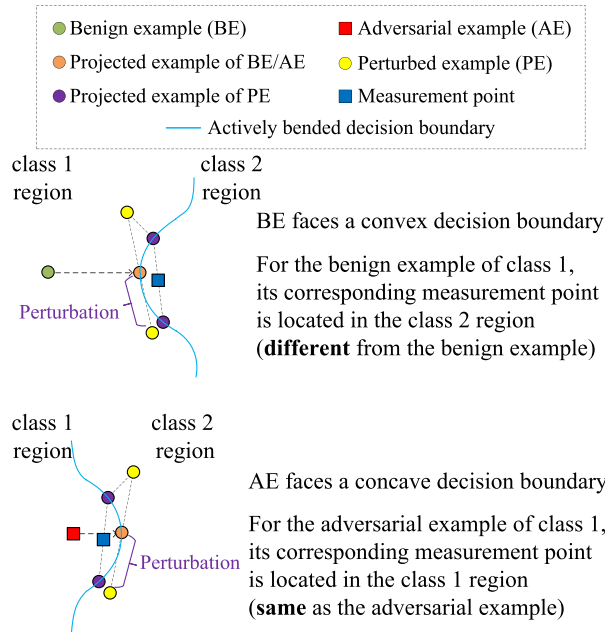


Fig. 3. Illustration of the projection-based auxiliary measurement method. Given a test example, it is first projected onto the decision boundary. Then, the projected example is perturbed in two directions (that are opposite to each other) and two perturbed examples are obtained. Next, the perturbed examples are re-projected onto the decision boundary to obtain a measurement point by calculating the midpoint of these corresponding projected examples. If the test example and the measurement point lie in two different classification regions, then the test example is identified as a benign example. Otherwise, if the test example and the measurement point lie in the same classification region, then the test example is identified as an SPA example.

without guaranteeing minimum perturbation magnitude. In this paper, we adopt five adversarial attacks (two general attacks and three small-perturbation adversarial attacks), which are listed below.

**Projected Gradient Descent (PGD) Attack:** The PGD attack [34] is an iterative method for generating adversarial examples belonging to gradient-based attacks. Formally, we

have

$$x'_{t+1} = \text{Clip}_{x,\epsilon}(x'_t + \epsilon \cdot \text{sign}(\nabla_{x'} \mathcal{L}(x', y))), \quad (1)$$

where  $t$  is the iteration number,  $\epsilon$  is the step size,  $\mathcal{L}(x', y)$  is the loss function,  $\nabla_{x'}$  means the gradient to the input  $x'$ , and  $\text{Clip}_{x,\epsilon}$  is a function that limits the perturbation between  $x - \epsilon$  and  $x + \epsilon$  and  $\text{sign}(\cdot)$  denotes the element-wise sign function.

**Carlini and Wagner (C&W) Attack:** The C&W attack [5] formulates an optimization problem to determine perturbations for input images. The optimization problem is summarized as

$$\min_{\delta} (\|\delta\|_p + c \cdot \mathcal{L}(x + \delta, y)), \quad (2)$$

where  $\|\delta\|_p$  is the perturbation magnitude,  $p$  is the norm used,  $\mathcal{L}(x + \delta, y)$  is the loss function, and  $c$  is a hyper-parameter to balance the perturbation and the loss.

**Fast Adaptive Boundary (FAB) Attack:** The FAB attack [6] employs an optimization-based approach to identify the minimal perturbation to shift an input example across the decision boundary into a different class. By iteratively refining the perturbation using gradient information, the attack determines the optimal modification direction. Ultimately, it induces misclassification with minimal distortion to the original example.

**Fast Minimum-Norm (FMN) Attack:** The FMN attack [40] leverages optimization techniques to generate adversarial examples with minimal perturbation. It operates by iteratively identifying the sample that is misclassified with the highest confidence, subject to an  $l_p$ -norm constant on the step size. Simultaneously, it adapts the step size to minimize the distance between the current sample and the decision boundary, ensuring both stable and efficient convergence.

**SuperDeepFool (SDF) Attack:** The SDF attack [1] introduces an additional projection step aiming to steer the direction of perturbation towards the optimal solution of finding the minimal perturbation. It maintains computational efficiency and is parameter-free while finding smaller adversarial perturbations.



## B. Adversarial Defenses

A variety of defense mechanisms have been proposed to mitigate the impact of adversarial attacks. Existing defense strategies can be divided into three types according to different stages of utilizing a model.

*Model Design-Phase Defenses:* During the model design period, defenses typically involve modifying the architecture of the original model. Xie et al. [53] developed new convolutional network architectures equipped with building blocks designed to denoise feature maps. Hendrik et al. [36] augmented a DNN with a small "detector" sub-network to distinguish benign data from AEs. Defenses in the model design period inevitably modify the model structure, leading to a degradation in performance on the original task.

*Model Training-Phase Defenses:* During the model training period, defenses typically involve training with novel approaches or using a new dataset to develop a more robust model, where adversarial training (AT) is the most popular method. Thermometer encoding [4] discretizes inputs while preserving relative distances. [12] proposes spectral norm normalization for convolutional layers. [56] proposes a joint defense combining phase-level adversarial training with amplitude-based pre-processing. SABRE [9] enhances robustness by using spectral decomposition and energy-based filtering to reconstruct inputs. Squeeze Training [28] improves robustness by jointly training on AEs and collaborative examples with low loss on both adversarial and benign inputs. These defenses can remain vulnerable to adaptive attacks when the attacker knows the defense mechanism.

*Model Inference-Phase Defenses:* During the model inference period, defenses typically involve processing input images instead of models, aiming to mitigate adversarial perturbations through denoising and purification techniques [10], [16], [20], [30]. Xie et al. [52] mitigate adversarial effects by randomly resizing inputs and applying random zero-padding. PixelDefend [44] purifies AEs by moving them back towards the high probability regions of the training distribution. ME-Net [55] randomly drops pixels from an input image and leverages matrix estimation to reconstruct the image.

During the model inference period, there are also plenty of detection approaches [17], [27], [31], [35], [45], [50]. They eliminate AEs by leveraging the intrinsic differences between normal examples and AEs. Feinman et al. [13] devised two features, named density estimates and Bayesian uncertainty estimates, to detect AEs. Feature squeezing [54], such as reducing the color bit depth and space smoothing, reduces the search space offered to an attacker. Local Intrinsic Dimensionality (LID) [33] measures the local space-filling capacity based on distances to neighboring points. Tian et al. [48] detected AEs by exploiting their sensitivity to perturbations near highly curved decision boundaries. These defenses always utilize conventional geometric features to detect AEs.

## III. PROBLEM FORMULATION

*Threat Model:* We consider two parties (the attacker and the defender) in the threat model.

TABLE I  
NOTATIONS

| Notation   | Description                                                   |
|------------|---------------------------------------------------------------|
| $x$        | input sample                                                  |
| $y$        | ground-truth label                                            |
| $f(\cdot)$ | a DNN classification model returning probability distribution |
| $D$        | original clean dataset                                        |
| $N$        | size of the dataset                                           |
| $r$        | perturbation ratio                                            |
| $d$        | minimum distance to the decision boundary                     |
| $\Delta$   | small perturbation                                            |
| $\delta$   | adversarial perturbation                                      |
| $\epsilon$ | adversarial step size                                         |
| $\kappa$   | principal curvature                                           |
| $\eta$     | random perturbation                                           |
| $t$        | threshold value used in the distance calculation stage        |

- **Attacker.** The goal of the attacker is to create SPA examples by adding small perturbations to the benign examples, making it hard for the defender to distinguish between SPA examples and benign examples. By feeding the SPA examples to the DNN, the predicted label is different from its original one. We consider that the attacker has white-box access to the target DNN. In the white-box access model, the attacker has all detailed knowledge about the DNN, including the model architecture, model parameters, etc. Note that a weaker access model is that the attacker has black-box access to the target DNN (i.e., can only query the model). The black-box attacker is weaker than the white-box attacker, so we mainly focus on the stronger white-box attacker in this paper.
- **Defender.** Given a dataset that consists of both SPA examples and benign examples, the defender aims to accurately detect SPA examples from the dataset. In addition, we consider that the defender is the DNN model owner who can train it.

*Design Goals:* PDBB should achieve the following design goals.

- **High Detection Accuracy.** Our approach PDBB should achieve a high true positive rate (TPR) and a low false positive rate (FPR) in detecting SPA examples.
- **High Clean Accuracy.** Compared with the original model (without bended boundaries), our approach PDBB should maintain comparable clean accuracy on benign inputs.
- **High Efficiency.** The computational overhead of training a model using PDBB should be similar to that of training the original model.

*Notations:* The commonly used notations in this paper are summarized in Table I.

## IV. METHODOLOGY

### A. Overview of PDBB

Fig. 1 illustrates the key idea of PDBB, which has been introduced in Section I. Fig. 2 overviews PDBB, which is composed

of two phases: 1) actively bending phase and 2) AE detection phase. The two phases are briefly described as follows.

1) *Actively Bending Phase*: In this phase, we develop the example-perturbation training strategy to train the target DNN model. In this strategy, for each class, we first slightly perturb a small portion of examples and assign them different labels. This perturbation strategy is applied to all label pairs to ensure coverage of all potential attack cases. Then, the perturbed examples are inserted into the original dataset, which is used to train the DNN, thereby achieving the desirable decision boundary bending. For the newly trained DNN, benign examples usually face a convex decision boundary in their vicinity. In contrast, SPA examples usually face a concave decision boundary in their vicinity.

2) *AE Detection Phase*. This phase contains two stages, as illustrated below.

i) *Distance Calculation Stage*. In this stage, PDBB identifies all examples that are located near the decision boundary. This is because SPA examples must be situated near the decision boundary. However, given a target example, it is challenging to calculate its minimum distance to the decision boundary. To address this issue, the FGSM attack [14] is employed to estimate the minimum perturbation that makes the target example be misclassified. Then, the minimum perturbation norm is used as an approximation of the target example's distance to the decision boundary. After the distance calculation, according to a pre-specified threshold, examples that lie near decision boundaries are identified. Next, the identified SPA example candidate set is forwarded to the next stage for further detection.

ii) *Concavity – Convexity Measurement Stage*. The second stage separates SPA examples from benign examples by comparing the concavity-convexity of the decision boundary around test examples. We propose a projection-based auxiliary measurement method to calculate the concavity-convexity of the decision boundary for a given test example in its vicinity. The details of the projection-based strategy are introduced in Section IV-B. There are two cases. If the test example faces a convex decision boundary in its vicinity, then it is identified as a benign example. If the test example faces a concave decision boundary in its vicinity, then it is identified as an SPA example.

## B. Detailed Design of PDBB

1) *Actively Bending Phase*: We design the following example-perturbation training strategy to train a target DNN model with the desirable bended decision boundary (referred to as the PDBB model). Let  $D = \{(x_i, y_i)\}_{i=1}^N$  denote the original clean dataset, where  $N$  is the dataset size.  $x_i$  denotes the clean input vector and  $y_i$  denotes the corresponding ground-truth label, which is from the label set  $Y$ . This phase consists of the following three steps.

- *Step 1 (Perturbation Pattern Design Step)*: Given the dataset  $D = \{(x_i, y_i)\}_{i=1}^N$ , we design  $|Y|$  different small perturbations, which are denoted as  $\Delta_l$  ( $l = 1, \dots, |Y|$ ). For perturbations, we use the same perturbation pattern, but put the pattern in different positions to form different perturbations. Note that when  $\Delta_l$  is imposed on an image, the label of the perturbed image will be set to  $l$ .

- *Step 2 (Perturbed Example Generation Step)*: We randomly choose a portion of the dataset from  $D = \{(x_i, y_i)\}_{i=1}^N$  (except the data with the label  $l$ ) to enforce the perturbation  $\Delta_l$  over them and set their label to be  $l$ . Then, we repeat the above process for  $l = 1, \dots, |Y|$ . The perturbed dataset is represented as  $\{(x'_i, y'_i)\}_{i=1}^M$ . Next, the perturbed dataset is inserted into the original dataset to form the perturbed training dataset  $D' = \{(x_i, y_i)\}_{i=1}^N \cup \{(x'_i, y'_i)\}_{i=1}^M$ . The perturbation ratio is defined as

$$r = \frac{M}{N}. \quad (3)$$

Note that the above perturbed example generation process considers all possible situations: different perturbed images of the same original category are assigned distinct labels corresponding to all other possible classes, ensuring comprehensive coverage of the label space. As a result, the DNN model trained over  $D'$  is the desirable model that bends all class-to-class decision boundaries.

- *Step 3 (Desirable DNN Training Step)*: We use  $D'$  to train the desirable DNN model with actively bended decision boundaries. The training process can be expressed as

$$\min_{\theta} \frac{1}{|D'|} \sum_{(x_i, y_i) \in D'} \mathcal{L}(y_i, f_{\theta}(x_i)), \quad (4)$$

where  $\theta$  are the model parameters.

2) *AE Detection Phase*: This phase consists of two stages, as elaborated below.

i) *Distance Calculation Stage*. This stage aims to calculate the minimum distance between a given test example and its nearby decision boundary. Accurately calculating the minimum distance is not an easy task. To address this problem, we propose the following method. First, we employ the FGSM attack to generate a minimal adversarial perturbation that causes the target example to be misclassified. Then, the  $l_2$  norm of the minimum perturbation is used as the approximated minimum distance (denoted as  $d$ ) between a given test example and its nearby decision boundary. The key reason behind this method is that the gradient descent method used in the FGSM attack can help to find the approximated minimum distance. Mathematically, we denote the input example as  $x$ , its ground-truth label as  $y$ , the adversarial example as  $x'$ , and the model outputting the probability distribution as  $f$ . The adversarial perturbation  $\delta$  is expressed as

$$\delta = \epsilon \cdot \text{sign}(\nabla_x \mathcal{L}(f(x), y)), \quad (5)$$

where  $\epsilon$  is the step size of the perturbation. The distance  $d$  can be formally expressed as

$$d = \|x' - x\|_2 = \|x + \delta - x\|_2 = \|\delta\|_2. \quad (6)$$

Algorithm 1 presents our algorithm to calculate the approximate minimum distance between a given test example and its nearby decision boundary. This algorithm iteratively searches for a minimum perturbation  $\epsilon$  such that the generated adversarial example is located sufficiently close to the decision boundary. Initially, the smallest step size  $\epsilon_0$  is set to 0, while the largest step size  $\epsilon_1$  is searched such that the perturbed example crosses the boundary. The  $\epsilon$  search procedure is illustrated in Lines 4

**Algorithm 1:** Distance Calculation.

---

**Input:** An image  $x$ , the corresponding ground-truth label  $y$ , the model  $f$ , the threshold value  $\tau = 0.05$ .

**Output:**  $d$

```

1  $\epsilon_0 \leftarrow 0$ ;
2 Search
    $\epsilon_1$  s.t.  $\arg \max(f(x + \epsilon_1 \cdot \text{sign}(\nabla_x \mathcal{L}(f(x), y)))) \neq y$ ;
3  $p_1 \leftarrow 1, p_2 \leftarrow 0$ ;
   /* The following loop stops until the
   adversarial example is
   sufficiently close to the decision
   boundary */
4 while  $p_1 - p_2 \geq \tau$  do
5    $\epsilon \leftarrow (\epsilon_0 + \epsilon_1)/2$ ;
6    $\delta \leftarrow \epsilon \cdot \text{sign}(\nabla_x \mathcal{L}(f(x), y))$ ;
7    $y' \leftarrow \arg \max(f(x + \delta))$ ;
8    $p_1, p_2 \leftarrow$  top-2 largest values in  $f(x + \delta)$ ;
9   if  $y' = y$  then
10     $\epsilon_0 \leftarrow \epsilon$ ;
    // increase  $\epsilon$  in the next
    iteration
11  else
12     $\epsilon_1 \leftarrow \epsilon$ ;
    // decrease  $\epsilon$  in the next
    iteration
13 return  $\|\delta\|_2$ ;
```

---

to 13, where  $\epsilon$  is updated as the average of  $\epsilon_0$  and  $\epsilon_1$ , while the two bounds are dynamically refined to gradually reduce the search space. In Line 9, if the label of the generated example is the same as  $y$ , which means that the perturbation magnitude is insufficient, the lower bound  $\epsilon_0$  is updated as the current  $\epsilon$ . In Line 11, if the label of the generated example differs from  $y$ , which means that the perturbation magnitude is too large, the upper bound  $\epsilon_1$  is updated as the current  $\epsilon$ . The search is considered successful when the generated example is close to the decision boundary (the difference between the top-2 largest probability values is smaller than  $\tau$ ).

ii) *Concavity-Convexity Measurement Stage*. In this stage, PDBB measures the concavity-convexity of the decision boundary around test examples to detect SPA examples. Fig. 3 illustrates how the projection-based auxiliary measurement method works. It consists of the following five steps.

- *Step 1 (First Projection Step)*: In this step, the test example  $x$  is projected onto the decision boundary. The projected example is denoted as  $\text{Proj}(x)$ , where the project function  $\text{Proj}$  is defined as follows.

**Definition 1:** (Projection Function) Given a classifier  $f$  that outputs softmax probabilities for each class, and an input  $x$ , let the top-2 predicted classes be

$$i = \arg \max_k f_k(x), \quad j = \arg \max_{k \neq i} f_k(x). \quad (7)$$

We define  $\text{Proj}(x)$  as the projection of  $x$  onto the decision boundary between class  $i$  and class  $j$ , i.e.,

$$\text{Proj}(x) = \arg \min_{x'} \|x - x'\|_2 \quad \text{s.t.} \quad f_i(x') \approx f_j(x'). \quad (8)$$

In practice,  $\text{Proj}(x)$  is implemented by using an efficient first-order procedure. Starting from the input  $x$ , we iteratively generate perturbed samples along the gradient direction of the cross-entropy loss with respect to the input:

$$x' = \text{clip}(x + \epsilon \cdot \text{sign}(\nabla_x \mathcal{L}_{CE}(x, y))), \quad (9)$$

where  $\mathcal{L}_{CE}$  denotes the cross-entropy loss and the clipping operation ensures a valid input range.

To approximate the minimal perturbation that reaches the decision boundary, the step size  $\epsilon$  is determined via a binary search over an interval  $[\epsilon_b, \epsilon_a]$ , where  $\epsilon_b = 0$ , and  $\epsilon_a$  is initialized by a grid search (with step size 0.01 up to 1.0) to find the smallest perturbation that causes misclassification.

We quantify the condition  $f_i(x') \approx f_j(x')$  using the *logit margin* between the top-1 and top-2 classes:

$$g(x') = z_{(1)}(x') - z_{(2)}(x'), \quad (10)$$

where  $z_{(1)}$  and  $z_{(2)}$  denote the largest and second-largest logits, respectively. The approximation “ $\approx$ ” is implemented as  $g(x') < 10^{-5}$ .

The procedure terminates when either the above condition is satisfied or a maximum of 300 iterations is reached. Among all candidates, we select the sample that minimizes  $g(x')$  as the final approximation of  $\text{Proj}(x)$ .

- *Step 2 (Perturbation Step)*: In this step, we perturb  $x'$  using opposite-direction perturbations to generate a pair of perturbed examples (PEs)

$$x'_1 = x' + \eta, \quad x'_2 = x' - \eta, \quad (11)$$

s.t.

$$\|\eta\|_1 = \gamma, \quad (12)$$

where  $\eta$  represents the random perturbation and  $\gamma$  is a constant. The use of opposite-direction perturbations aims to ensure that the generated example pairs are distributed around  $x'$ , allowing for a more accurate characterization of the local structure of the decision boundary in the following steps.

- *Step 3 (Second Projection Step)*: In this step, PDBB projects the perturbed example pairs  $x'_1, x'_2$  generated in Step 2 onto the decision boundary. The projected examples are denoted as  $x''_1$  and  $x''_2$ , respectively. This step enables PDBB to get the measurement point used in the next step.
- *Step 4 (Measurement Point Determination Step)*: In this step, the middle point of these projected examples of PE is considered as the measurement point. Mathematically, the measurement point  $m$  is calculated by

$$m = \frac{x''_1 + x''_2}{2}. \quad (13)$$

- *Step 5 (Concavity-Convexity Calculation Step)*: If  $x$  and  $m$  lie in two different classification regions, then the

**Algorithm 2:** Projection-Based Auxiliary Measurement.

---

**Input:** An image  $x$ , the corresponding ground-truth label  $y$ , the number of iterations  $T$ , the model  $f$

**Output:** Concave or Convex. // Convex represents BE, whereas concave represents AE

```

1  $x' \leftarrow \text{Proj}(x)$ ;
  /* Proj() denotes the projection of
  examples onto the decision
  boundary */
2  $sum \leftarrow 0$ ;
3 for  $i \leftarrow 1$  to  $T$  do
4   sample  $\eta$  such that  $\|\eta\|_1 = \gamma$ ;
  // perturbation
5    $x'_1 \leftarrow x' + \eta, x'_2 \leftarrow x' - \eta$ ;
6    $P \leftarrow f(\frac{\text{Proj}(x'_1) + \text{Proj}(x'_2)}{2})$ ;
  /*  $P$  represents the probability
  distribution of the measurement
  point */
7    $p_1, p_2 \leftarrow \text{top-2 largest values in } P$ ;
8    $y' \leftarrow \arg \max(P)$ ;
9   if  $y' = y$  then
10     $sum \leftarrow sum + p_1 - p_2$ ;
11  else
12     $sum \leftarrow sum + p_2 - p_1$ ;
13  $avg \leftarrow sum/T$ ;
14 if  $avg > 0$  then
15   return Concave;
16 else
17   return Convex;
```

---

test example  $x$  is identified as a benign example. Otherwise, if  $x$  and  $m$  lie in the same classification region, then the test example  $x$  is identified as an SPA example.

Remarkably, to enhance the reliability of the results, PDBB reaches the final decision by running Step 2 to Step 5 multiple times.

The pseudo-code of the projection-based auxiliary measurement method is exhibited in Algorithm 2. Line 1 represents Step 1. Lines 4-5 represent Step 2. Line 6 represents Step 3 and Step 4. Lines 7-17 represent Step 5.

## V. ANALYSIS OF PDBB

### A. Feasibility Analysis of PDBB

In this section, we provide a theoretical analysis of why the concavity-convexity measurement in PDBB can be used to recognize SPA examples. Specifically, Claim 1 proves that the decision boundary in the vicinity of well-crafted SPA examples is usually concave. In addition, Claim 2 proves that using the model with actively bended boundaries (i.e., PDBB model) can enhance the well-crafted SPA example detection performance of PDBB.

**Definition 2:** (Margin Function) Let  $f(x)$  be the probability output of a classifier. Let  $y$  be the ground-truth label of  $x$ . We define the margin function as

$$M(x) = f_y(x) - \max_{y' \neq y} f_{y'}(x), \quad (14)$$

where  $f_y(x)$  is the probability for being predicted as  $y$ , and the second term is the highest probability among all predicted labels  $y' \neq y$ .

**Definition 3:** (Decision Boundary) Let  $f(x)$  be the probability output of a classifier, the decision boundary between classes  $i$  and  $j$  is defined as

$$\mathcal{B}_{i,j} = \{x | f_i(x) - f_j(x) = 0\}. \quad (15)$$

**Definition 4:** (Well-Crafted SPA Example) A well-crafted SPA example  $x_{adv} = x + \delta$  is defined as an adversarial example generated by a small perturbation  $\delta$  which crosses the decision boundary along the shortest path in input space. Such examples are subject to the following two properties.

- The perturbation direction is approximately orthogonal to the gradient of the margin function at  $x_{adv}$ ,

$$\nabla M(x_{adv})^\top v \approx 0. \quad (16)$$

- Within a small range  $\alpha \in (0, \epsilon)$  in the direction  $v$  from  $x_{adv}$ , the margin function continues to decrease, i.e.,

$$M(x_{adv} + \alpha v) < 0, \quad \forall \alpha \in (0, \epsilon). \quad (17)$$

If an SPA example fails to satisfy these two properties, it implies that the example does not cross the decision boundary along the shortest path and the perturbation is large, thereby reducing its threat level. In this section, we focus on analyzing the feasibility of PDBB against well-crafted SPA examples (see the following claim).

**Claim 1:** Let  $x_{adv} = x + \delta$  be a well-crafted SPA example misclassified from class  $y$  to  $y'$  where  $\delta$  is the adversarial perturbation, and let  $v = \delta/\|\delta\|$  be the normalized direction of the perturbation. We have

$$v^\top \nabla^2 M(x_{adv}) v < 0, \quad (18)$$

which means that the decision boundary between class  $y$  and  $y'$  is locally concave in the vicinity of  $x_{adv}$ .

**Proof:** According to (14), the decision boundary between class  $y$  and  $y'$  is defined by  $M(x) = 0$ . At  $x_{adv}$ ,  $M(x_{adv}) < 0$ .

Let  $\alpha v$  be a perturbation that continues in the direction of the original adversarial perturbation, where  $\alpha$  is the step size. We expand  $M(x_{adv} + \alpha v)$  as

$$\begin{aligned} M(x_{adv} + \alpha v) &= M(x_{adv}) + \alpha \nabla M(x_{adv})^\top v \\ &\quad + \frac{1}{2} \alpha^2 v^\top \nabla^2 M(x_{adv}) v. \end{aligned} \quad (19)$$

According to Definition 4, the perturbation direction is approximately orthogonal to the gradient of the margin function at  $x_{adv}$ , where  $\nabla M(x_{adv})^\top v \approx 0$ . Therefore, whether  $M(x_{adv} + \alpha v)$  becomes more negative depends mainly on  $\alpha^2 v^\top \nabla^2 M(x_{adv}) v$ .

According to Definition 4, within a small range  $\alpha \in (0, \epsilon)$  in the direction  $v$  from  $x_{adv}$ ,  $M(x_{adv} + \alpha v) < 0$ . Then, in order



for  $M(x)$  to continue decreasing along  $v$ , it must hold that

$$v^\top \nabla^2 M(x_{adv}) v < 0. \quad (20)$$

Thus, the decision boundary between class  $y$  and  $y'$  is locally concave in the vicinity of  $x_{adv}$  along the direction  $v$ .  $\square$

*Claim 2:* Let  $x_c$  be a benign example and  $x_{adv} = x_c + \delta$  be a corresponding well-crafted adversarial example such that  $\arg \max f(x_c) = y$ ,  $\arg \max f(x_{adv}) = y'$ . Let  $M(x)$  be the margin function and  $v = \delta / \|\delta\|$  be the normalized direction of the perturbation.

Then, using the PDBB model, the probability that the decision boundary  $\mathcal{B}_{y,y'}$  in the vicinity of  $x_{adv}$  has negative curvature in perturbation direction  $v$  is higher, i.e.,

$$P[v^\top \nabla^2 M(x_{adv}) v < 0] > P[v^\top \nabla^2 M(x_c) v < 0]. \quad (21)$$

*Proof:* When using actively perturbed examples to train a target DNN, examples  $x + \Delta_l$  from class  $y \neq l$  are labeled as class  $l$ , and the model is optimized to satisfy

$$f_l(x + \Delta_l) > f_y(x + \Delta_l), \forall l \neq y, \quad (22)$$

which implies that  $M(x + \Delta_l) = f_y(x + \Delta_l) - f_l(x + \Delta_l) < 0$ .

To achieve this during training, the model must reduce  $M$  in a neighborhood around  $x + \Delta_l$ . Let  $v_\Delta$  denote the direction from the clean sample  $x$  to its perturbed version  $x + \Delta_l$ .

Given that the gradient  $\nabla M(x + \Delta_l)$  is not strongly aligned with  $v_\Delta$ , i.e.,  $\nabla M(x + \Delta_l)^\top v_\Delta \approx 0$  because  $\Delta_l$  does not follow the direction of the gradient descent, we can approximate the Taylor expansion of the margin along  $v_\Delta$  as

$$M(x + \Delta_l + \epsilon v_\Delta) \approx M(x + \Delta_l) + \frac{1}{2} \epsilon^2 v_\Delta^\top \nabla^2 M(x + \Delta_l) v_\Delta. \quad (23)$$

Considering that the margin continues to decrease along  $v_\Delta$  because  $\Delta_l$  is constructed to reduce the margin  $M$ , then this behavior implies that

$$v_\Delta^\top \nabla^2 M(x + \Delta_l) v_\Delta < 0. \quad (24)$$

That is, the margin function is concave in direction  $v_\Delta$ , and the boundary  $\mathcal{B}_{y,l}$  bends toward the class  $y$  near  $x + \Delta_l$ .

The decision boundary bending creates a shortcut in the geometry of the boundary. According to Definition 4,  $x_{adv}$  crosses the decision boundary along the shortest path, and so  $x_{adv}$  is located in the region near  $x + \Delta_l$ , where

$$x_{adv} \approx x + \Delta_l. \quad (25)$$

Due to the decision boundary bending induced by actively perturbed examples,  $M(x)$  exhibits negative curvature along the perturbation direction at  $x_{adv}$ , as established in Claim 1, i.e.,

$$v^\top \nabla^2 M(x_{adv}) v < 0. \quad (26)$$

In contrast,  $x_c$  typically lie near the center of the class  $y$  where the influence of the decision boundary bending is small. Thus,

$$P[v^\top \nabla^2 M(x_{adv}) v < 0] > P[v^\top \nabla^2 M(x_c) v < 0]. \quad (27)$$

$\square$

According to Claim 2, the decision boundary in the vicinity of SPA examples using the PDBB model is more likely to be

TABLE II  
ATTACK CONFIGURATIONS

| Attack Method | Attack Configuration                                                                   |
|---------------|----------------------------------------------------------------------------------------|
| PGD           | max eps = 0.01, # of iteration = 100, eps of each iteration = 0.0001                   |
| C&W           | binary step size = 9, max iterations = 10000, learning rate = 0.01, abort early = True |
| FAB           | # of iteration = 100, max alpha = 0.1                                                  |
| FMN           | binary step size = 10, # of iteration = 100                                            |
| SDF           | # of iteration = 100                                                                   |

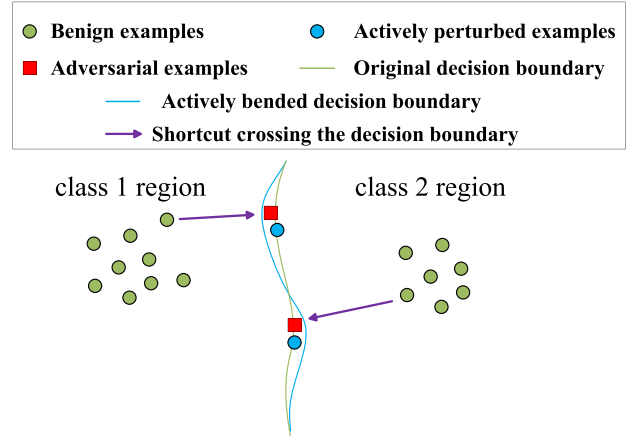


Fig. 4. An example to illustrate why there are no boundary conflicts. The shortcut from class 1 to class 2 (for benign examples in class 1) and the shortcut from class 2 to class 1 (for benign examples in class 2) are in different locations.

concave compared to that using the original clean model. As a result, it is feasible to utilize PDBB to detect SPA examples utilizing the concavity-convexity of the decision boundary.

### B. Analysis of Extra Training Cost

To ensure comprehensive coverage of the label space, different perturbed images of the same original category are assigned distinct labels corresponding to all other possible classes. The total number of perturbed examples  $M$  can be expressed as

$$M = |Y| \cdot (|Y| - 1), \quad (28)$$

where  $Y$  denotes the label set and  $|Y|$  is the number of classes. The additional number of perturbed examples grows quadratically with the number of classes. For example, when  $|Y| = 10$  (e.g., CIFAR-10),  $M = 90$ , corresponding to approximately 0.18% of the original training dataset, which introduces only negligible additional cost. When  $|Y| = 1000$  (e.g., ImageNet-1K),  $M = 9.99 \cdot 10^5$ , corresponding to roughly 78% of the original training dataset, leading to a substantial increase in training cost. This increase in computational cost is comparable to standard data augmentation or pairwise training and remains acceptable in practice.

### C. Analysis on Why There are no Boundary Conflicts

Boundary conflicts in PDBB mean that the active bending operations may bend the decision boundary at the same locations for different label pairs. Fig. 4 shows an example to illustrate why there are no boundary conflicts when using two types of



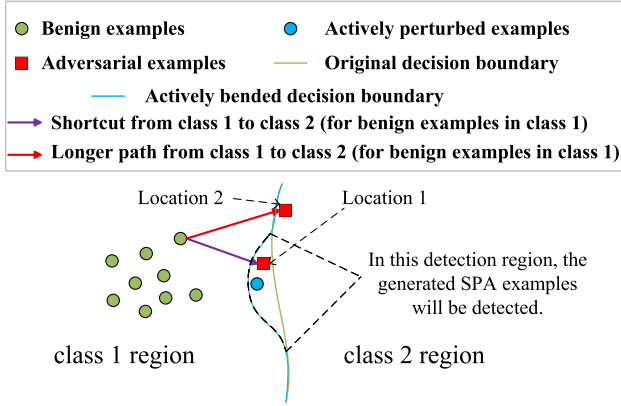


Fig. 5. An example to show why PDBB can mitigate adaptive attacks. The benign example in Location 1 has to move to Location 2 along a longer path to escape PDBB detection. Therefore, PDBB forces an SPA example to have a large perturbation, thereby reducing the attack stealthiness.

label pairs: 1) the original label is 1 and the attack target label is 2, and 2) the original label is 2 and the attack target label is 1. PDBB needs to bend the decision boundary to capture the above two types of SPA attacks. Through our experiments, it is found that the bended decision boundaries are located in two different locations in the high-dimensional space. As shown in Fig. 4, the shortcut from class 1 to class 2 (for benign examples in class 1) and the shortcut from class 2 to class 1 (for benign examples in class 2) are in different locations. Therefore, there are no boundary conflicts in PDBB.

#### D. Analysis on Mitigating Adaptive Attacks

Adaptive Attacks refer to attacks in which the attacker knows how PDBB works. Fig. 5 shows why PDBB can mitigate adaptive attacks. To bypass PDBB, the adaptive attacker has to generate SPA examples that are not located in the detection region (see Fig. 5). Therefore, the generated SPA examples have to move along a longer path to cross the decision boundary. As shown in Fig. 5, the benign example in Location 1 has to move to Location 2 along a longer path to escape PDBB detection. Therefore, PDBB forces an SPA example to have a large perturbation, thereby reducing the attack stealthiness. Note that an SPA example with a larger perturbation is easier to observe by defenders, so PDBB can mitigate adaptive attacks. In Section VI-B6, we implement an adaptive FAB attack and conduct experiments to measure the perturbation magnitude. Our experimental results confirm our analysis.

## VI. EXPERIMENTS

### A. Experimental Setup

**Datasets:** We conduct experiments on four popular datasets.

- Hand-written Digit Recognition (MNIST) [26] - a benchmark collection of gray-scale images of handwritten digits, labeled from 0 to 9, commonly used for evaluating machine learning models in image classification tasks.
- Clothes Recognition (Fashion-MNIST) [51] - a dataset of gray-scale images of clothing items across 10 categories,

designed as a more challenging alternative to the original MNIST dataset.

- Image Recognition (CIFAR-10) [24] - a widely used collection of color images divided into 10 classes, representing everyday objects such as animals, vehicles, and tools, commonly used for adversarial defense literature.
- Street View House Numbers (SVHN) [38] - a real-world image dataset containing color images of house number digits extracted from Google Street View.

**Parameter Settings:** When evaluating PDBB on CIFAR-10 and SVHN datasets with color images, we use a standard ResNet-18 [18] model, a standard ResNet-34 [18] model, and a DenseNet-121 [19] model. For the MNIST and Fashion-MNIST datasets with gray-scale images, we use LeNet [25] model. All neural networks are trained using the Adam optimizer [22] for 100 epochs, with an initial learning rate of 0.001 that is decayed by a factor of 0.1 at epochs 30, 60, and 90. In the proposed PDBB, the deployment-time hyper-parameters ( $\tau$ ,  $\gamma$ ,  $r$ , and  $T$ ) are not selected using labeled adversarial examples. Instead, they are determined based on benign validation data and remain fixed during deployment.

**Metrics:** The used metrics are listed below.

- Clean Accuracy ( $CA$ ) is the classification accuracy of a model evaluated on clean data.
- Detection Accuracy ( $DAcc$ ) is the fraction of adversarial and benign examples that are correctly detected.
- True Positive Rate ( $TPR$ ) is the fraction of adversarial examples that are correctly detected.
- False Positive Rate ( $FPR$ ) is the fraction of benign examples that are incorrectly detected as adversarial.
- $F_1$ -score is the metric in classification tasks that combines both precision and recall into a single value. The  $F_1$ -score is defined as

$$F_1 = \frac{2 \times Precision \times Recall}{Precision + Recall}.$$

- Area Under the Curve ( $AUC$ ) is the metric that quantifies the ability of a model to distinguish between the positive and negative classes across all possible classification thresholds.

**Adversarial Attack Configuration:** We evaluate PDBB using two general adversarial attacks (PGD and C&W attack) and three SPA attacks (FAB, FMN, and SDF attack). Among the evaluated attacks, FAB, FMN, and SDF are representative SPA attacks since they explicitly optimize for minimum-norm perturbations. PGD and C&W are additionally included to evaluate the effectiveness of PDBB under more general adversarial settings. In addition, we evaluate PDBB using a variant of AutoAttack [7], referred to as SPA AutoAttack (SPA-AA), which consists of FAB, FMN, SDF, and Square attacks [2]. More details about our attack configuration are in Table II. We choose a small constant for the step size  $\epsilon$  and incorporate an early-stopping mechanism into the PGD attack to improve its efficiency. For the remaining attacks, the default parameters are sufficient to generate effective adversarial examples, so we adopt their default settings in our evaluation. Nearly all attacks against the PDBB models achieve a success rate above 95%. The success rate of the PGD attack

TABLE III  
 CLEAN ACCURACY OF THE CLEAN MODEL AND PDBB MODEL TRAINED ON VARIOUS DATASETS

| Dataset     | MNIST  | F-MNIST | CIFAR-10 (ResNet-18) | CIFAR-10 (DenseNet-121) | SVHN (ResNet-18) | SVHN (DenseNet-121) | Average |
|-------------|--------|---------|----------------------|-------------------------|------------------|---------------------|---------|
| Clean model | 99.45% | 92.29%  | 90.67%               | 94.00%                  | 96.21%           | 97.07%              | 94.95%  |
| PDBB model  | 99.45% | 91.93%  | 91.25%               | 94.46%                  | 95.77%           | 97.10%              | 94.99%  |

is about 40% because of the small perturbation. We randomly select 3,000 benign examples from the original dataset and generate their corresponding 3,000 adversarial examples to evaluate the performance of PDBB.

**Baseline Methods:** We use the following baseline methods to compare with our PDBB.

- **Local Intrinsic Dimensionality (LID)** [33]. This method identifies AEs by analyzing the distance distribution of an example's neighbors.
- **Mahalanobis Distance (MD)** [27]. This method identifies AEs by measuring the probability density of test examples on the feature spaces of DNNs.
- **Sensitivity Inconsistency Detector (SID)** [48]. This method identifies AEs by using the difference in sensitivity between normal examples and AEs to perturbations in highly curved regions of the decision boundary.

In implementing these baseline methods, we adopt the default parameters and settings provided in their original papers.

**Implementation:** All programming codes are implemented in PyTorch [39]. All our experiments are conducted on an ML workstation with one NVIDIA GeForce RTX 4090 GPU.

## B. Experimental Results

**1) PDBB Model Performance:** We report the PDBB model performance in terms of  $CA$ ,  $DAcc$ ,  $TPR$ ,  $FPR$ ,  $F_1$ -score,  $AUC$ , and training cost below.

- **Clean Accuracy.** Table III shows the clean accuracy of the clean and PDBB model trained on different datasets. We find that the accuracy of PDBB models remains comparable to that of clean models across all datasets, suggesting that bending the decision boundary leads to nearly no clean accuracy degradation. Notably, the PDBB model achieves an accuracy of 94.46% on CIFAR-10 (DenseNet-121), outperforming the clean model with an accuracy of 94.00%. This may be attributed to the fact that bending the decision boundary increases the dataset size by incorporating actively perturbed examples, thereby providing the model with more opportunities to learn original image features.

Fig. 6 displays the clean accuracy of clean and PDBB models by varying epochs. For both models, the clean accuracy is increasing with an increasing training epoch. In addition, the clean accuracy of the PDBB model remains comparable to that of the clean model.

- **$DAcc$  and  $AUC$ .** Table IV shows the  $DAcc$  and  $AUC$  of PDBB evaluated across various datasets, models, and attacks. It can be found that PDBB has a strong detection capability across diverse datasets. For example, on a ResNet-18 model trained on CIFAR-10, PDBB achieves the highest detection accuracy among all compared methods, averaging 99.08%.

- **Other Performance Metrics.** Table V shows the  $TPR$ ,  $FPR$ , and  $F_1$ -score of PDBB detection for different datasets, models, and attacks. We have three findings. First, PDBB

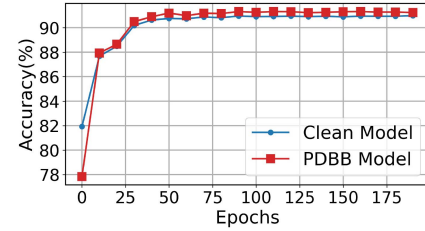


Fig. 6. Classification accuracy of clean and PDBB models by varying epochs. ResNet-18 and CIFAR-10 are used in experiments.

exhibits strong capability in recognizing SPA examples. For example, in most cases, the  $TPR$  for PDBB exceeds 95.00% and the average  $F_1$ -score remains above 98.00%. In addition, the  $FPR$  remains below 1.00% while maintaining a high  $TPR$ , indicating that PDBB effectively distinguishes benign examples from SPA ones. Second, PDBB shows great robustness across different model architectures. It holds that  $DAcc < 2\%$  across different models on CIFAR-10 and SVHN datasets. Third, PDBB can be effectively employed to defend against ensembling SPA attack strategies. When confronted with SPA AutoAttack, PDBB also exhibits good detection performance, with the average  $TPR$  of 98.00% and  $F_1$ -score of 98.65%.

- **Training Cost.** The additional computational overhead introduced by PDBB is in an acceptable range. As shown in Table VI, the average per-epoch training time of the clean model and the PDBB model on CIFAR-10 and SVHN is 11.89 s and 12.95 s, indicating an 8.9% increase.

**2) Performance Compared With Baseline Defenses:** Table IV compares PDBB with baseline methods. There are three critical findings.

First, PDBB outperforms most baseline methods in terms of detection performance. For example, when evaluated on the ResNet-18 model trained on CIFAR-10, the average detection accuracy of LID, MD, and SID is 75.01%, 79.74%, and 86.00%, respectively, which is lower than that of PDBB by 24.07%, 19.34%, and 13.08%. The average  $AUC$  is also lower than that of PDBB by 13.75%, 9.23%, and 8.51%, respectively. SPA attacks induce small changes in the feature space, making baseline methods harder to detect.

Second, PDBB exhibits strong generalization ability across different datasets and model architectures. For instance, on SVHN, PDBB achieves an average detection accuracy of 99.35% on the ResNet-18 model, which is similar to the average accuracies of 99.10% and 99.12% achieved on the ResNet-34 and DenseNet-121 models. Similarly, on SVHN, PDBB achieves comparable  $AUC$  scores across different model architectures, with average values of 95.43%, 95.63%, and 95.68%, respectively.

Third, PDBB exhibits more stable detection performance compared to other detection methods, showing its robustness.

TABLE IV  
COMPARISON OF OUR PDBB WITH BASELINE METHODS LID, MD, AND SID USING DIFFERENT MODELS AND DATASETS

| Dataset       | Model       | Methods | $D_{Acc}$     |               |               |               |               | $AUC$         |               |               |               |               |
|---------------|-------------|---------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
|               |             |         | PGD           | C&W           | FAB           | FMN           | SDF           | PGD           | C&W           | FAB           | FMN           | SDF           |
| CIFAR-10      | ResNet18    | LID     | 69.40%        | 75.95%        | 80.92%        | 70.44%        | 78.36%        | 76.45%        | 83.59%        | 87.96%        | 77.96%        | 85.07%        |
|               |             | MD      | 71.62%        | 89.85%        | 80.49%        | 76.04%        | 80.70%        | 78.44%        | <b>95.50%</b> | 87.22%        | 82.91%        | 87.44%        |
|               |             | SID     | 61.86%        | 89.07%        | 92.87%        | 94.72%        | 91.48%        | 58.60%        | 94.00%        | 96.22%        | 98.13%        | 94.95%        |
|               |             | PDBB    | <b>98.80%</b> | <b>98.50%</b> | <b>99.60%</b> | <b>99.40%</b> | <b>99.10%</b> | <b>90.45%</b> | 92.43%        | <b>98.78%</b> | <b>99.31%</b> | <b>96.17%</b> |
|               | ResNet34    | LID     | 68.57%        | 74.79%        | 82.33%        | 73.29%        | 79.95%        | 74.44%        | 82.76%        | 89.31%        | 80.83%        | 87.38%        |
|               |             | MD      | 73.97%        | 95.40%        | 81.19%        | 84.84%        | 81.96%        | <b>98.94%</b> | 88.44%        | 91.43%        | 88.95%        | 88.95%        |
|               |             | SID     | 60.54%        | 88.45%        | 91.33%        | 94.95%        | 92.40%        | 60.54%        | 94.41%        | 95.44%        | 97.72%        | <b>96.72%</b> |
|               |             | PDBB    | <b>95.80%</b> | <b>96.20%</b> | <b>97.80%</b> | <b>97.70%</b> | <b>97.10%</b> | <b>93.40%</b> | 94.77%        | <b>96.21%</b> | <b>98.55%</b> | 95.20%        |
|               | DenseNet121 | LID     | 69.40%        | 84.07%        | 77.45%        | 80.01%        | 77.46%        | 75.74%        | 91.96%        | 83.46%        | 88.04%        | 84.32%        |
|               |             | MD      | 71.82%        | 93.26%        | 59.41%        | 88.04%        | 56.34%        | 79.19%        | <b>97.87%</b> | 63.07%        | 94.24%        | 58.91%        |
|               |             | SID     | 61.11%        | 91.64%        | 88.00%        | 89.81%        | 86.06%        | 41.61%        | 94.78%        | 92.89%        | 94.94%        | 91.85%        |
|               |             | PDBB    | <b>94.00%</b> | <b>97.60%</b> | <b>99.34%</b> | <b>95.50%</b> | <b>98.40%</b> | <b>90.34%</b> | 95.96%        | <b>97.85%</b> | <b>95.74%</b> | <b>98.50%</b> |
| SVHN          | ResNet18    | LID     | 68.87%        | 81.68%        | 89.28%        | 80.47%        | 88.91%        | 75.73%        | 89.51%        | 94.17%        | 87.85%        | 93.94%        |
|               |             | MD      | 78.44%        | 92.64%        | 81.44%        | 76.98%        | 81.61%        | 86.28%        | 97.16%        | 88.43%        | 84.34%        | 88.76%        |
|               |             | SID     | 51.89%        | 95.35%        | 95.81%        | 97.11%        | 96.72%        | 41.30%        | 98.25%        | 98.23%        | 98.94%        | <b>98.79%</b> |
|               |             | PDBB    | <b>99.40%</b> | <b>99.40%</b> | <b>99.30%</b> | <b>99.30%</b> | <b>98.70%</b> | <b>99.65%</b> | <b>99.80%</b> | <b>98.45%</b> | <b>99.81%</b> | 95.20%        |
|               | ResNet34    | LID     | 66.35%        | 84.78%        | 87.47%        | 81.33%        | 87.50%        | 71.31%        | 92.30%        | 94.12%        | 88.94%        | 94.10%        |
|               |             | MD      | 74.22%        | 94.85%        | 77.56%        | 83.06%        | 78.29%        | 80.41%        | 98.57%        | 85.04%        | 90.65%        | 85.74%        |
|               |             | SID     | 70.22%        | 95.69%        | 95.29%        | 97.42%        | <b>97.87%</b> | 61.49%        | 98.30%        | 98.22%        | <b>99.21%</b> | <b>98.99%</b> |
|               |             | PDBB    | <b>99.55%</b> | <b>97.75%</b> | <b>99.61%</b> | <b>99.50%</b> | <b>97.20%</b> | <b>94.40%</b> | <b>99.36%</b> | <b>99.93%</b> | 99.20%        | 96.80%        |
|               | DenseNet121 | LID     | 69.73%        | 85.84%        | 85.24%        | 80.10%        | 84.80%        | 76.35%        | 93.38%        | 92.03%        | 88.08%        | 91.76%        |
|               |             | MD      | 76.66%        | 95.41%        | 71.98%        | 83.67%        | 69.39%        | 85.95%        | <b>98.86%</b> | 79.33%        | 91.21%        | 76.41%        |
|               |             | SID     | 62.50%        | 91.93%        | 81.39%        | 95.38%        | 93.62%        | 34.38%        | 96.24%        | 88.11%        | 98.55%        | <b>97.48%</b> |
|               |             | PDBB    | <b>98.90%</b> | <b>99.19%</b> | <b>98.90%</b> | <b>99.40%</b> | <b>99.20%</b> | <b>92.97%</b> | 98.56%        | <b>95.60%</b> | <b>99.60%</b> | 95.03%        |
| MNIST         | LeNet       | LID     | 52.25%        | 65.61%        | 54.76%        | 55.98%        | 51.95%        | 52.07%        | 71.64%        | 56.21%        | 58.04%        | 52.41%        |
|               |             | MD      | 54.60%        | 80.44%        | 70.85%        | 59.65%        | 55.70%        | 55.47%        | 88.21%        | 76.60%        | 62.61%        | 57.85%        |
|               |             | SID     | 61.90%        | 98.34%        | 97.43%        | 98.32%        | 92.42%        | 59.03%        | <b>99.76%</b> | <b>99.34%</b> | 99.68%        | <b>96.95%</b> |
|               |             | PDBB    | <b>98.25%</b> | <b>99.50%</b> | <b>99.31%</b> | <b>99.60%</b> | <b>92.60%</b> | <b>99.20%</b> | 96.51%        | 98.81%        | <b>99.80%</b> | 84.60%        |
| Fashion-MNIST | LeNet       | LID     | 50.84%        | 51.07%        | 50.84%        | 52.38%        | 50.34%        | 50.83%        | 51.00%        | 49.96%        | 52.65%        | 50.28%        |
|               |             | MD      | 52.62%        | 52.09%        | 50.37%        | 51.93%        | 50.58%        | 53.38%        | 51.49%        | 50.06%        | 52.04%        | 50.55%        |
|               |             | SID     | 60.91%        | 91.27%        | 93.64%        | 92.16%        | 93.08%        | 60.23%        | <b>95.86%</b> | <b>97.70%</b> | 96.93%        | <b>96.34%</b> |
|               |             | PDBB    | <b>97.90%</b> | <b>99.00%</b> | <b>99.20%</b> | <b>98.30%</b> | <b>93.30%</b> | <b>93.24%</b> | 94.85%        | 97.66%        | <b>99.52%</b> | 92.66%        |

For example, while MD achieves an average detection accuracy of approximately 92.84% and an  $AUC$  of 97.44% against the C&W attack on CIFAR-10, its performance drops significantly to 73.70% in average accuracy and 79.58% in  $AUC$  against the FAB attack. In contrast, the detection accuracy and  $AUC$  of PDBB on the CIFAR-10 dataset consistently stay above 90% across different attacks.

3) *Performance in Detecting General AEs*: As shown in Table IV, compared with baseline methods, PDBB exhibits similar performance in detecting general AEs. Therefore, PDBB can still work well on general AEs with relatively larger distances from the decision boundaries.

4) *Impact of Critical Parameters*: In our experiments, unless otherwise stated, when we vary one parameter, we keep all other parameters at their default values.

*Impact of  $\epsilon$* : As illustrated in Fig. 7, we observe that PDBB generally outperforms baseline methods in terms of  $AUC$ , regardless of the perturbation step size  $\epsilon$ .

*Impact of  $\gamma$* : Fig. 8 shows the impact of  $\ell_1$ -norm constant  $\gamma$  (See (12)) on the detection performance. With an increasing  $\gamma$ , the detection performance is first increasing first and then

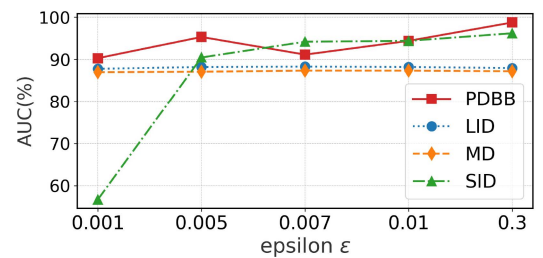


Fig. 7. The  $AUC$  scores of PDBB, LID, MD, and SID with varying the perturbation step size  $\epsilon$  using FAB attack on CIFAR-10 dataset.

decreasing. When  $\gamma = 0.01$ , the best detection performance can be observed, achieving 99.20% in TPR, 99.50% in  $F1$ -score, and 98.80% in  $AUC$ .

*Impact of  $r$* : Table VII shows the influence of the perturbation ratio  $r$  (See (3)) on the detection performance of PDBB. Compared with no boundary bending case (i.e.,  $r = 0$ ), using proactive boundary bending (i.e.,  $r > 0$ ) can always help SPA examples detection. In addition, setting  $r = 0.9\%$  achieves the



TABLE V  
 PDBB PERFORMANCE FOR VARIOUS DATASETS, MODELS AND ATTACKS

| Dataset       | Model       | Attack | TPR   | FPR  | $F_1$ -score |
|---------------|-------------|--------|-------|------|--------------|
| CIFAR-10      | ResNet18    | PGD    | 97.6% | 0.4% | 98.59%       |
|               |             | C&W    | 98.0% | 1.0% | 98.50%       |
|               |             | FAB    | 99.2% | 0.2% | 99.50%       |
|               |             | FMN    | 93.7% | 0.5% | 96.51%       |
|               |             | SDF    | 98.8% | 0.6% | 99.10%       |
|               |             | SPA-AA | 98.4% | 1.2% | 98.60%       |
|               | ResNet34    | PGD    | 95.8% | 1.4% | 97.18%       |
|               |             | C&W    | 93.8% | 1.4% | 96.14%       |
|               |             | FAB    | 96.4% | 0.8% | 97.78%       |
|               |             | FMN    | 95.4% | 0.4% | 97.45%       |
|               |             | SDF    | 94.6% | 0.4% | 97.04%       |
|               |             | SPA-AA | 96.8% | 1.4% | 97.69%       |
|               | DenseNet121 | PGD    | 92.8% | 1.6% | 95.52%       |
|               |             | C&W    | 97.1% | 1.1% | 97.99%       |
|               |             | FAB    | 98.2% | 1.1% | 98.55%       |
|               |             | FMN    | 94.4% | 1.0% | 96.65%       |
|               |             | SDF    | 97.0% | 0.2% | 98.38%       |
|               |             | SPA-AA | 98.2% | 0.7% | 98.75%       |
| SVHN          | ResNet18    | PGD    | 98.8% | 0.3% | 99.25%       |
|               |             | C&W    | 98.8% | 0%   | 99.40%       |
|               |             | FAB    | 98.6% | 0.2% | 99.20%       |
|               |             | FMN    | 98.6% | 0%   | 99.30%       |
|               |             | SDF    | 97.6% | 0.2% | 98.69%       |
|               |             | SPA-AA | 99.6% | 0.4% | 99.60%       |
|               | ResNet34    | PGD    | 99.1% | 0.5% | 99.30%       |
|               |             | C&W    | 95.5% | 1.0% | 97.22%       |
|               |             | FAB    | 99.4% | 0.2% | 99.60%       |
|               |             | FMN    | 99.0% | 0.2% | 99.40%       |
|               |             | SDF    | 94.4% | 0%   | 97.12%       |
|               |             | SPA-AA | 97.0% | 0.6% | 98.19%       |
|               | DenseNet121 | PGD    | 98.6% | 0.8% | 98.90%       |
|               |             | C&W    | 98.2% | 0.3% | 98.94%       |
|               |             | FAB    | 98.5% | 0.5% | 99.00%       |
|               |             | FMN    | 99.2% | 0.4% | 99.40%       |
|               |             | SDF    | 99.0% | 0.6% | 99.20%       |
|               |             | SPA-AA | 99.6% | 0.2% | 99.70%       |
| MNIST         | LeNet       | PGD    | 96.5% | 0.2% | 98.12%       |
|               |             | C&W    | 99.0% | 1.0% | 99.00%       |
|               |             | FAB    | 98.6% | 0%   | 99.30%       |
|               |             | FMN    | 99.6% | 0.4% | 99.60%       |
|               |             | SDF    | 85.4% | 0.2% | 92.04%       |
|               |             | SPA-AA | 99.0% | 0%   | 99.50%       |
| Fashion-MNIST | LeNet       | PGD    | 95.8% | 0.6% | 97.57%       |
|               |             | C&W    | 98.0% | 0%   | 98.99%       |
|               |             | FAB    | 97.0% | 0.4% | 98.28%       |
|               |             | FMN    | 96.6% | 0%   | 98.27%       |
|               |             | SDF    | 87.4% | 0.8% | 92.93%       |
|               |             | SPA-AA | 95.4% | 1.0% | 97.17%       |

 TABLE VI  
 ONE-EPOCH TRAINING TIME CONSUMPTION OF THE CLEAN MODEL AND PDBB MODEL TRAINED ON VARIOUS DATASETS. (S: SECOND)

| Model       | ResNet-18 |        | ResNet-34 |        | Average |
|-------------|-----------|--------|-----------|--------|---------|
| Dataset     | CIFAR-10  | SVHN   | CIFAR-10  | SVHN   |         |
| Clean model | 7.22s     | 10.55s | 12.20s    | 17.57s | 11.89s  |
| PDBB model  | 7.86s     | 11.40s | 13.28s    | 19.26s | 12.95s  |

best detection performance. Therefore, a slight  $r = 0.9\%$  increasing of the training dataset is sufficient for our purpose.

*Impact of  $T$ :* Table VIII shows the influence of  $T$  in the concavity-convexity measurement stage on the detection performance. It is observed that increasing the number of iterations has a positive effect on detection performance.

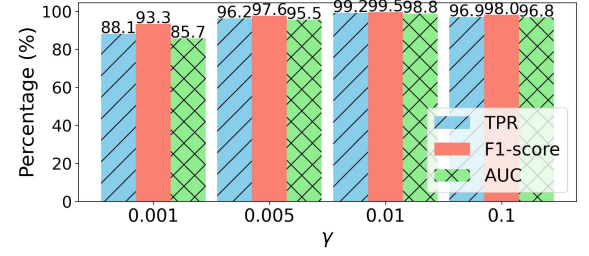


Fig. 8. Detection results of PDBB on a ResNet-34 model under the FAB attack with different  $\gamma$  in concavity-convexity measurement stage.

 TABLE VII  
 PDBB PERFORMANCE ON A RESNET-18 MODEL UNDER THE FAB ATTACK, TRAINED WITH DIFFERENT RATIO  $r$ 

| Ratio $r$ | TPR           | FPR    | $F_1$ -score  | AUC           |
|-----------|---------------|--------|---------------|---------------|
| 0%        | 97.34%        | 15.00% | 90.72%        | 93.10%        |
| 0.9%      | <b>98.63%</b> | 0.60%  | <b>99.01%</b> | <b>99.04%</b> |
| 4.5%      | 97.91%        | 1.00%  | 98.45%        | 95.83%        |
| 9.0%      | 97.21%        | 0.80%  | 98.19%        | 96.64%        |
| 13.5%     | 97.52%        | 1.00%  | 98.25%        | 95.39%        |

 TABLE VIII  
 PDBB PERFORMANCE ON A RESNET-18 MODEL UNDER THE FAB ATTACK UNDER DIFFERENT NUMBER OF ITERATIONS  $T$ 

| Iteration $T$ | TPR    | FPR   | $F_1$ -score | AUC    |
|---------------|--------|-------|--------------|--------|
| 1             | 97.71% | 1.00% | 98.35%       | 97.44% |
| 3             | 97.74% | 1.80% | 97.97%       | 96.16% |
| 5             | 99.20% | 0.20% | 99.51%       | 99.50% |
| 7             | 98.86% | 0.60% | 99.13%       | 98.92% |
| 10            | 97.73% | 1.80% | 98.25%       | 97.96% |

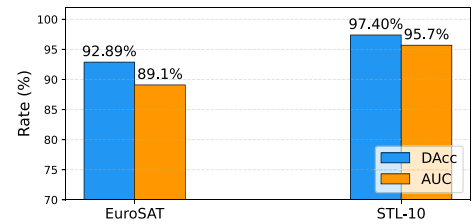


Fig. 9. Detection performance on unseen datasets using the same hyper-parameter configuration without dataset-specific tuning.

To further evaluate the robustness and stability of the selected hyper-parameters, we additionally conducted experiments on two previously unseen datasets, EuroSAT (satellite images from 10 land-use classes) and STL-10 (higher-resolution natural images from 10 object classes), using exactly the same parameter settings as those adopted in the original experiments, without any dataset-specific retuning. The results demonstrate consistently strong detection performance across different data domains, as summarized in Fig. 9. These observations indicate that the proposed method is relatively insensitive to moderate hyper-parameter variations and that the parameter configuration in one dataset can generalize effectively across multiple datasets without requiring labeled adversarial samples.



TABLE IX  
RESULTS OF THE ABLATION STUDY

| Variations of PDBB                       | TPR           | $F_1$ -score  | AUC           |
|------------------------------------------|---------------|---------------|---------------|
| PDBB w/o decision boundary bending       | 97.34%        | 90.72%        | 93.10%        |
| PDBB w/o distance calculation            | 97.71%        | 81.56%        | 96.48%        |
| PDBB w/o concavity-convexity measurement | 99.40%        | 94.90%        | -             |
| PDBB                                     | <b>99.20%</b> | <b>99.50%</b> | <b>98.78%</b> |

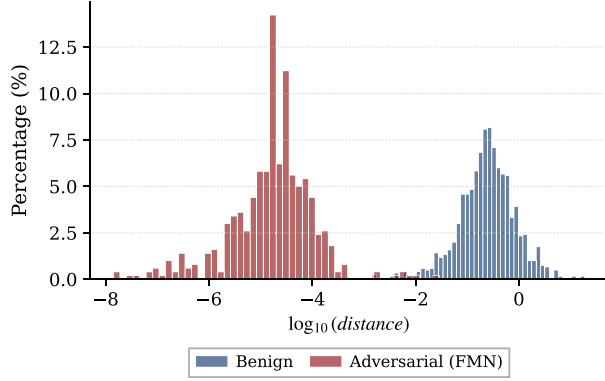


Fig. 10. The distribution of the estimated distance  $d$  for benign and SPA examples.

5) *Ablation Study*: To investigate the contribution of each stage in PDBB, we evaluate its detection performance under conditions where one individual component is removed, and the result is shown in Table IX. There are three findings. First, all components contribute to the detection performance. Second, PDBB w/o concavity-convexity measurement reduces the  $F_1$ -score, with a decrease of 4.60%. Therefore, concavity-convexity measurement indeed contributes to SPA detection. Third, considering that the lack of decision boundary bending leads to a reduction in  $F_1$ -score and AUC, decision boundary bending helps to boost the detection performance of PDBB.

To further understand the contribution of the distance calculation stage, we analyze the distribution of the estimated distance  $d$  for benign and SPA examples, as shown in Fig. 10. We observe a separation between the two distributions. Specifically, benign examples tend to have larger distance values. In contrast, SPA examples exhibit significantly smaller distances. This distributional difference explains the effectiveness of the first-stage filtering. By applying distance calculation, a large portion of SPA examples can be filtered out while preserving most benign examples. This also aligns with the ablation results in Table IX, where removing the distance calculation leads to a substantial drop in  $F_1$ -score.

6) *Performance on Adaptive Attacks*: We have devised and implemented an adaptive FAB attack in our experiments. Specifically, the objective of the adaptive FAB attack is modified as

$$S_c = D_c - \lambda_d(p_1 - p_2) - \lambda_c M(m), \quad (29)$$

where  $D_c$  is the original FAB distance metric,  $(p_1 - p_2)$  is the top-1/top-2 confidence margin used as a surrogate for the first-stage distance filter, and  $M(m)$  denotes the classification margin

TABLE X  
PERTURBATION STATISTICS OF FAB ATTACK UNDER STANDARD AND ADAPTIVE SETTINGS

| Attack         | Metric        | Mean    | Median  | $P01$   | $P99$   |
|----------------|---------------|---------|---------|---------|---------|
| FAB            | $\ell_2$      | 0.1718  | 0.1549  | 0.0064  | 0.4488  |
|                | $\ell_\infty$ | 0.00335 | 0.00302 | 0.00012 | 0.00946 |
| FAB (Adaptive) | $\ell_2$      | 0.2714  | 0.2130  | 0.00915 | 0.8287  |
|                | $\ell_\infty$ | 0.00579 | 0.00396 | 0.00017 | 0.02035 |

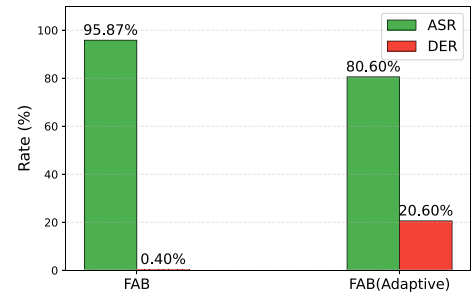


Fig. 11. The comparison between FAB and adaptive FAB attack. ASR: attack success rate; DER: detection evasion rate (fraction of successful adversarial examples that evade detection).

at the measurement point  $m$  used by the second-stage concavity-convexity detector.

We have two groups of experiments to explain the performance of adaptive attack. The results when the perturbation budget is not restricted are shown in Table X.  $P01$  in the table denotes the 1st percentile, meaning that only 1% of the examples have smaller values than this threshold.  $P99$  in the table denotes the 99th percentile, meaning that 99% of the examples have smaller values than this threshold. We observe that the adaptive attack remains effective but incurs a significant increase in perturbation magnitude. Specifically, the mean  $\ell_2$  norm increases from 0.1718 to 0.2714 (approximately +58%), and the mean  $\ell_\infty$  norm increases from 0.00335 to 0.00579 (approximately +73%). Similar trends are observed for the median and high-percentile values. These results suggest that, although the adaptive attack can be successful, it leads to a larger perturbation of the SPA example. Note that an SPA example with a larger perturbation is easier to observe by defenders, so PDBB can mitigate adaptive attacks.

The experiment where the FAB attack and adaptive FAB attack are under nearly the same perturbation budget is conducted. The results shown in Fig. 11 confirm that incorporating detector-aware objectives substantially increases the difficulty of generating successful adversarial examples. Compared with standard FAB attack, Adaptive FAB attack exhibits a lower ASR (80.60% versus 95.87%), suggesting that simultaneously optimizing for attack success and detector evasion constitutes a more challenging optimization problem. Although the DER increases from 0.40% to 20.60%, indicating that the adaptive objectives can partially improve evasion capability, PDBB still detects 79.40% of the successful adversarial examples generated by Adaptive FAB attack. These findings suggest that evading PDBB requires a non-negligible sacrifice in attack effectiveness,

highlighting the robustness of PDBB against detector-aware adaptive attacks.

## VII. CONCLUSION

In this paper, we have developed PDBB, which helps in detecting SPA examples via proactive decision boundary bending. Unlike prior passive solutions, PDBB takes a proactive posture in SPA example detection. The defender actively modifies the target model to facilitate SPA example detection. It is demonstrated that PDBB not only achieves the SOTA performance in detecting SPA examples but also achieves high performance for detecting general AEs. Remarkably, this research demonstrates the feasibility of proactive solutions in SPA example detection. The exploration of better proactive solutions is a promising future research direction.

## REFERENCES

- [1] M. Abroshan et al., "SuperDeepFool: A new fast and accurate minimal adversarial attack," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 98537–98562.
- [2] M. Andriushchenko, F. Croce, N. Flammarion, and M. Hein, "Square attack: A query-efficient black-box adversarial attack via random search," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 484–501.
- [3] M. Bojarski et al., "End to end learning for self-driving cars," 2016, *arXiv:1604.07316*.
- [4] J. Buckman, A. Roy, C. Raffel, and I. Goodfellow, "Thermometer encoding: One hot way to resist adversarial examples," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 75–89.
- [5] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 39–57.
- [6] F. Croce and M. Hein, "Minimally distorted adversarial examples with a fast adaptive boundary attack," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 2196–2205.
- [7] F. Croce and M. Hein, "Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 2206–2216.
- [8] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 4690–4699.
- [9] A. F. Diallo and P. Patrás, "Sabre: Cutting through adversarial noise with adaptive spectral filtering and input reconstruction," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 2901–2919.
- [10] G. K. Dziugaite, Z. Ghahramani, and D. M. Roy, "A study of the effect of JPG compression on adversarial images," 2016, *arXiv:1608.00853*.
- [11] A. Esteva et al., "Dermatologist-level classification of skin cancer with deep neural networks," *Nature*, vol. 542, pp. 115–118, 2017.
- [12] F. Farnia, J. Zhang, and D. Tse, "Generalizable adversarial training via spectral normalization," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–27.
- [13] R. Feinman, R. R. Curtin, S. Shintre, and A. B. Gardner, "Detecting adversarial samples from artifacts," 2017, *arXiv:1703.00410*.
- [14] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2014, *arXiv:1412.6572*.
- [15] K. Grosse, P. Manoharan, N. Papernot, M. Backes, and P. McDaniel, "On the (Statistical) detection of adversarial examples," 2017, *arXiv:1702.06280*.
- [16] C. Guo, M. Rana, M. Cisse, and L. van der Maaten, "Countering adversarial images using input transformations," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–12.
- [17] P. Harder, F.-J. Pfrendt, M. Keuper, and J. Keuper, "SpectralDefense: Detecting adversarial attacks on CNNs in the fourier domain," in *Proc. Int. Joint Conf. Neural Netw.*, 2021, pp. 1–8.
- [18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [19] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 4700–4708.
- [20] Y. Huang et al., "AdvFilter: Predictive perturbation-aware filtering against adversarial attack via multi-domain learning," in *Proc. ACM Int. Conf. Multimedia*, 2021, pp. 395–403.
- [21] A. Ilyas, S. Santurkar, D. Tsipras, L. Engstrom, B. Tran, and A. Madry, "Adversarial examples are not bugs, they are features," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 125–136.
- [22] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
- [23] B. R. Kiran et al., "Deep reinforcement learning for autonomous driving: A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 6, pp. 4909–4926, Jun. 2022.
- [24] A. Krizhevsky et al., "Learning multiple layers of features from tiny images," Univ. Toronto, Toronto, ON, Canada, Tech. Rep. TR-2009, 2009.
- [25] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," in *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [26] Y. LeCun, C. Cortes, and C. J. C. Burges, "The MNIST database of handwritten digits," AT&T Labs. [Online]. Available: <https://yann.lecun.com/exdb/mnist/>
- [27] K. Lee, K. Lee, H. Lee, and J. Shin, "A simple unified framework for detecting out-of-distribution samples and adversarial attacks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 7167–7177.
- [28] Q. Li, Y. Guo, W. Zuo, and H. Chen, "Squeeze training for adversarial robustness," in *Proc. Int. Conf. Learn. Representations*, 2023, pp. 1–16.
- [29] X. Li and F. Li, "Adversarial examples detection in deep networks with convolutional filter statistics," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 5764–5772.
- [30] F. Liao, M. Liang, Y. Dong, T. Pang, X. Hu, and J. Zhu, "Defense against adversarial attacks using high-level representation guided denoiser," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 1778–1787.
- [31] H. Liu, B. Zhao, M. Ji, Y. Peng, J. Guo, and P. Liu, "Feature-filter: Detecting adversarial examples by filtering out recessive features," *Appl. Soft Comput.*, vol. 124, 2022, Art. no. 109027.
- [32] J. Liu et al., "Detection based defense against adversarial examples from the steganalysis point of view," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 4825–4834.
- [33] X. Ma et al., "Characterizing adversarial subspaces using local intrinsic dimensionality," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–15.
- [34] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–10.
- [35] D. Meng and H. Chen, "MagNet: A two-pronged defense against adversarial examples," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 135–147.
- [36] J. H. Metzen, T. Genewein, V. Fischer, and B. Bischoff, "On detecting adversarial perturbations," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–12.
- [37] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, "DeepFool: A simple and accurate method to fool deep neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2574–2582.
- [38] Y. Netzer et al., "Reading digits in natural images with unsupervised feature learning," in *Proc. NIPS Workshop Deep Learn. Unsupervised Feature Learn.*, 2011, Art. no. 4.
- [39] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 8024–8035.
- [40] M. Pintor, F. Roli, W. Brendel, and B. Biggio, "Fast minimum-norm adversarial attacks through adaptive norm constraints," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 20052–20062.
- [41] A. Prakash, N. Moran, S. Garber, A. DiLillo, and J. Storer, "Deflecting adversarial attacks with pixel deflection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 8571–8580.
- [42] A. Raghunathan, J. Steinhardt, and P. Liang, "Certified defenses against adversarial examples," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 11447–11456.
- [43] A. Robey, F. Latorre, G. J. Pappas, H. Hassani, and V. Cevher, "Adversarial training should be cast as a non-zero-sum game," 2023, *arXiv:2306.11035*.
- [44] Y. Song, S. Nowozin, N. Kushman, T. Kim, and S. Ermon, "PixelDefend: Leveraging generative models to understand and defend against adversarial examples," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–20.
- [45] P. Sperl, C.-Y. Kao, P. Chen, X. Lei, and K. Böttinger, "DLA: Dense-layer-analysis for adversarial example detection," in *Proc. IEEE Eur. Symp. Secur. Privacy*, 2020, pp. 198–215.

- [46] C. Szegedy et al., "Intriguing properties of neural networks," 2013, *arXiv:1312.6199*.
- [47] Y. Taigman, M. Yang, M. Ranzato, and L. Wolf, "DeepFace: Closing the gap to human-level performance in face verification," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2014, pp. 1701–1708.
- [48] J. Tian, J. Zhou, Y. Li, and J. Duan, "Detecting adversarial examples from sensitivity inconsistency of spatial-transform domain," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 9877–9885.
- [49] E. Wong and Z. Kolter, "Provable defenses against adversarial examples via the convex outer adversarial polytope," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5286–5295.
- [50] Y. Wu, S. S. Arora, Y. Wu, and H. Yang, "Beating attackers at their own games: Adversarial example detection using adversarial gradient directions," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 2969–2977.
- [51] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.
- [52] C. Xie, J. Wang, Z. Zhang, Z. Ren, and A. Yuille, "Mitigating adversarial effects through randomization," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–16.
- [53] C. Xie, Y. Wu, L. van der Maaten, A. L. Yuille, and K. He, "Feature denoising for improving adversarial robustness," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 501–509.
- [54] W. Xu, D. Evans, and Y. Qi, "Feature squeezing: Detecting adversarial examples in deep neural networks," in *Proc. Netw. Distrib. Syst. Symp.*, 2018, pp. 1–15.
- [55] Y. Yang, G. Zhang, D. Katabi, and Z. Xu, "ME-Net: Towards effective adversarial robustness with matrix estimation," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 7025–7034.
- [56] D. Zhou, N. Wang, H. Yang, X. Gao, and T. Liu, "Phase-aware adversarial defense for improving adversarial robustness," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 42724–42741.



**Nankun Mu** received the BS degree from the School of Big Data & Software Engineering, Chongqing University, Chongqing, China, in 2011, and the PhD degree from the College of Computer Science, Chongqing University, Chongqing, China, in 2015. He is currently an associate professor with the College of Computer Science, Chongqing University, Chongqing, China. His research interests include AI security, data trading, privacy protection, and intelligent optimization.



**Qi Xia** received the bachelor's degree in computer science and technology from Hohai University, Nanjing, China, in 2023, and the master's degree from the College of Computer Science, Chongqing University, Chongqing, China, in 2026. His research interests include artificial intelligence security and large language model.



**Fengyi Jiang** received the bachelor's degree from the Software College, Northeastern University, Shenyang, China, in 2021, and the master's degree from the College of Computer Science, Chongqing University, Chongqing, China, in 2026. His current research focuses on AI security and LLM.



**Hongyu Huang** (Member, IEEE) received the PhD degree from the Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai, China, in 2009. He is currently an associate professor with the College of Computer Science, Chongqing University, Chongqing, China. His current research focuses on cybersecurity, deep learning, and AI4Science.



**Di Zhang** received the PhD degree in intelligent computing and information processing from Southwest University, Chongqing, China, in 2021. From 2018 to 2020, she was a visiting scholar with Virginia Polytechnic Institute, Arlington, VA, USA. She is currently an associate professor with the College of Computer Science, Chongqing University, Chongqing, China. Her current research interests include applied crypto, privacy protection, blockchain, and AI security.



**Xiaowei Yang** received the BS degree from the School of Mechanical and Electrical Engineering, Shandong Jianzhu University, Shandong, China, in 2012, and the PhD degree from the School of Automation Science and Electrical Engineering, Beihang University, Beijing, China, in 2020. He is currently a senior engineer with the China Industrial Control Systems Cyber Emergency Response Team, Beijing, China. His research interests include AI security, network security, data security, data trading, and privacy protection.



**Bei Gong** (Member, IEEE) received the BS degree from Shandong University, in 2005, and the PhD degree from the Beijing University of Technology, in 2012. In the past five years, he has published more than 30 papers in first-class SCI/EI and other international famous journals and top international conferences in relevant research fields. His research interests include trusted computing, Internet of Things security, Industrial Internet of Things, and privacy data protection.



**Weizhi Meng** (Senior Member, IEEE) is a full professor with the School of Computing and Communications, Lancaster University, United Kingdom, and an adjunct faculty with the Department of Applied Mathematics and Computer Science, Technical University of Denmark, Denmark. He received the IEEE ComSoc Best Young Researcher Award for Europe, Middle East, & Africa Region (EMEA) in 2020. His primary research interests include intersections among cyber security, artificial intelligence and blockchain technology, such as intrusion detection, IoT security, biometric authentication, and blockchain. He serves as associate editors / editorial board members for many reputed journals such as the *IEEE Transactions on Dependable and Secure Computing* and *IEEE Transactions on Information Forensics and Security*, as well as chair for various international conferences such as ACM CCS'23 and ESORICS'22.



**Xinyu Lei** (Member, IEEE) received the PhD degree from the Department of Computer Science and Engineering, Michigan State University, East Lansing, MI, USA, in 2021. He is currently an assistant professor with the Department of Computer Science, Michigan Technological University, Houghton, MI, USA. He worked as a research assistant with the Texas A&M University at Qatar, Doha, Qatar, in 2013. His current research focuses on machine learning and cybersecurity.